

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«Херсонский государственный педагогический университет»

Институт информационных технологий

Кафедра программной инженерии и информационных технологий

КУРСОВАЯ РАБОТА

**на тему: «Разработка сайта-визитки
фуллстек-разработчика»**

**по дисциплине «Современные операционные системы
и базы данных»**

Обучающегося 2 курса группы 12-25РПм Направления подготовки 09.04.04

Трифорова Д.С.

Научный руководитель:

Галлини Н.И.

Оглавление

ВВЕДЕНИЕ	2
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ САЙТА-ВИЗИТКИ	4
1.1 Сайт-визитка как форма цифрового представления специалиста	4
1.2 Клиент-серверная модель и роль операционной системы	4
1.3 Хранение данных в веб-приложениях	5
2 ПРОЕКТИРОВАНИЕ САЙТА-ВИЗИТКИ	7
2.1 Требования к функциональности и качеству	7
2.2 Выбор технологического стека	7
2.3 Информационная архитектура и интерфейс	8
3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И РАЗВЁРТЫВАНИЕ	9
3.1 Структура проекта и основные компоненты	9
3.2 Взаимодействие с операционной средой хостинга	9
3.3 Клиентское хранение настроек и вычисляемые данные	10
3.4 Тестирование и результаты	10
ЗАКЛЮЧЕНИЕ	11
Список использованных источников	12

ВВЕДЕНИЕ

В профессиональной среде первого впечатления часто достаточно нескольких секунд просмотра персональной страницы. Прежде чем состоится разговор или переписка, заинтересованная сторона уже успевает оценить, как специалист оформляет своё присутствие в сети, насколько ясно изложены сильные стороны и насколько удобно начать контакт. Компактная персональная страница, которую принято называть визиткой, как раз и решает эту задачу: она собирает ключевые сведения в одном экране и задаёт визуальный тон.

Если речь идёт о фуллстек-разработчике, ценность такого ресурса возрастает. Страница одновременно показывает практический навык вёрстки и клиентской логики, аккуратность в подготовке к выкладке и умение держать интерфейс понятным на разных устройствах. Даже когда объём контента невелик, приходится продумать иерархию блоков, скорость открытия, устойчивость вёрстки и порядок файлов на сервере.

Тема курсовой работы актуальна потому, что веб-страница существует не «сама по себе», а внутри связки «браузер - сеть - операционная среда хостинга». Для дисциплины «Современные операционные системы и базы данных» здесь важны как минимум три аспекта: как ОС обслуживает выдачу файлов, где физически лежат данные и чем клиентское хранилище отличается от серверной СУБД. Отдельно учитывается подготовка ресурса к индексации.

Цель работы - создать и подробно описать персональную визитку фуллстек-разработчика, пояснив выбранный стек и схему размещения в сети.

Чтобы достичь цели, последовательно решаются следующие задачи:

- выяснить, какие ожидания предъявляются к персональной веб-визитке специалиста;
- разобраться, как браузер и сервер взаимодействуют при открытии страницы и какую роль здесь играет ОС;
- сопоставить варианты хранения сведений: файлы на диске, браузерное хранилище и классическая СУБД;
- спроектировать и собрать одностраничный интерфейс с адаптацией и умеренной интерактивностью;
- выложить результат на хостинг и оформить минимальный набор средств поисковой подготовки.

Объект исследования - создание небольшого персонального веб-ресурса специалиста.

Предмет исследования - приёмы построения сайта-визитки фуллстек-разработчика с опорой на свойства операционных систем и способы хранения данных.

В работе применяются изучение источников и документации, сопоставление вариантов архитектуры, макетирование интерфейса, практическая реализация, проверка в браузерах и публикация на хостинге.

Практический итог - действующая визитка по адресу <http://trifonix.beget.tech/>. Полученные решения можно переносить на другие лёгкие персональные страницы.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ САЙТА-ВИЗИТКИ

1.1 Сайт-визитка как форма цифрового представления специалиста

Под сайтом-визиткой понимают небольшой публичный ресурс, где человек или организация кратко представляются аудитории [1]. У него нет задач полноценного портала или торговой площадки: не нужны корзина, личный кабинет и сложные сценарии. На первый план выходят ясная подача, быстрый отклик страницы и понятный путь к контакту.

В персональной визитке разработчика обычно встречаются:

- блок идентичности: ФИО, снимок, должность или роль;
- короткое резюме умений;
- сведения о местах работы и учёбе;
- список технологий;
- выходы на проекты и мессенджеры.

С инженерной позиции такой ресурс ближе к информационной витрине, чем к транзакционной системе. Контент меняется редко, поэтому его удобно держать в разметке. Вместе с тем на клиенте допустимы небольшие сценарии - смена оформления, подсчёт стажа, анимация акцентов. Главное остаётся прежним: показать сведения, а не обслуживать поток операций.

Для фуллстек-разработчика визитка ещё и «пробный стенд» полного пути поставки: сверстать, оформить стилями, добавить логику, положить файлы на сервер, проверить служебные `robots.txt` и `sitemap.xml`, убедиться в корректном виде на телефоне и мониторе. В итоге страница работает и как продукт, и как фрагмент портфолио.

1.2 Клиент-серверная модель и роль операционной системы

Сеть веб-ресурсов опирается на разделение ролей клиента и сервера [2]. Клиент - это браузер: он отправляет запрос по HTTP или HTTPS, получает ответ, разбирает разметку, применяет таблицы стилей, запускает сценарии и рисует интерфейс. Сервер принимает обращение, находит нужный файл либо формирует ответ программно и возвращает его обратно.

ОС на стороне сервера создаёт условия для работы веб-сервиса (часто Apache либо Nginx): поднимает процессы, контролирует права на каталоги, открывает сетевые порты, ведёт журналы, обслуживает сертификаты [3]. Даже если страница полностью статическая, без корректной настройки ОС нельзя гарантировать безопасный доступ к публичным файлам и стабильную выдачу.

На устройстве пользователя своя ОС (Windows, macOS, Android, iOS и другие) запускает браузер как приложение, выделяет ему память и сеть, ограничивает доступ к системе. Внутри браузера действуют собственные правила: исполнение JavaScript преимущественно в одном потоке интерфейса, политика same-origin, лимиты на объём `localStorage`. Эти ограничения нужно учитывать уже на этапе проектирования.

Открытие визитки обычно проходит так:

1. человек набирает адрес или переходит по ссылке;
2. служба имён преобразует домен в IP;
3. устанавливается защищённое соединение;
4. сервер под управлением ОС отдаёт `index.html` и связанные файлы;
5. браузер строит дерево документа, накладывает стили и выполняет скрипты.

Зная эту цепочку, проще выбрать площадку размещения: shared-хостинг, VPS или чисто статический хост. В курсовом проекте использован виртуальный хостинг Beget: ОС и веб-сервер предоставляет площадка, а автор отвечает за наполнение открытого каталога.

1.3 Хранение данных в веб-приложениях

СУБД нужны там, где сведения нужно систематически сохранять, искать и изменять [4]. В типичном динамическом сайте серверная база (PostgreSQL, MySQL и аналоги) держит пользователей, материалы и события; приложение читает и пишет её через программный слой, а ОС обеспечивает процессы СУБД и доступ к дискам.

Для визитки отдельный серверный склад часто избыточен: тексты и изображения зафиксированы и могут жить прямо в файлах проекта. При этом хранение всё равно проявляется в двух плоскостях.

На сервере файловая система ОС хранит страницу, иллюстрации, иконки, манифест и SEO-служебники. Здесь важны порядок каталогов, типы содержимого и правила кэша.

В браузере доступен Web Storage. Через `localStorage` можно оставлять строковые пары ключ-значение между визитами [5]. В отличие от cookie, эти записи не уезжают на сервер с каждым запросом. Реляционных выборок, транзакций и совместного доступа нескольких пользователей там нет, зато удобно помнить личные настройки оформления.

В выполненном проекте в `localStorage` фиксируется выбранная палитра (светлая либо тёмная). Так разделяются:

- содержимое самой визитки, которое сопровождает исходный код;
- предпочтения посетителя, привязанные к его браузеру.

Отдельно стоит отметить машиночитаемые описания Open Graph и JSON-LD. Это не замена СУБД, но способ сообщить поисковикам и мессенджерам, о ком страница и какое изображение к ней относится.

2 ПРОЕКТИРОВАНИЕ САЙТА-ВИЗИТКИ

2.1 Требования к функциональности и качеству

Перед сборкой интерфейса зафиксирован перечень ожидаемых возможностей:

1. показать фото, ФИО, роль и короткое описание умений;
2. вывести места работы и образование;
3. перечислить ключевые технологии;
4. дать анонс собственного SaaS и ссылку на него;
5. обеспечить быстрый выход на почту, GitHub и Telegram;
6. позволить менять светлую и тёмную палитру с запоминанием выбора;
7. автоматически считать срок работы «по настоящее время»;
8. уложить материал в макет «экран смартфона» так, чтобы основной каркас не требовал прокрутки.

К качеству предъявлялись требования лаконичного внешнего вида, быстрого открытия, понятных элементов управления, готовности к выкладке и базовой поисковой подготовки (title, description, canonical, Open Graph, sitemap).

Условие «один экран» заставило собрать сведения мозаикой внутри рамки телефона. Такой каркас усиливает ощущение мобильного формата и делает композицию собранной.

2.2 Выбор технологического стека

Реализация опирается на HTML5, CSS3 и JavaScript без серверных каркасов. Выбор следует из характера задачи: тексты почти не меняются, реакции происходят локально, а выкладка на обычный хостинг должна оставаться простой.

Разметка отвечает за смысл страницы: главный заголовок с именем, заголовки блоков, контактные ссылки и служебные метаданные. Стили строят сетку, неоновую эстетику, темы через CSS-переменные и движения элементов. Сценарии отвечают за стартовый экран загрузки, подсчёт стажа, смену темы и лёгкий параллакс фона.

Отказ от серверной СУБД на текущем шаге не уводит работу в сторону от дисциплины. Напротив, в тексте явно показано:

- где хватает файлов под контролем ОС;
- где уместно браузерное хранилище;

- когда без полноценной базы уже не обойтись (учётки, CMS, комментарии, поток событий).

В перспективе можно добавить API и таблицу проектов, чтобы обновлять сведения без правки HTML. Для учебной цели хватает статической поставки, и она проще для демонстрации идей курса.

2.3 Информационная архитектура и интерфейс

Страница собрана из одной мозаики блоков:

- верхняя полоса - личность, статус, контакты;
- опыт - две актуальные позиции с датами;
- образование - вуз, направление, ссылка на сайт;
- навыки - набор компактных «плашек»;
- SaaS - краткий анонс DigiGu.

Визуально всё помещено в корпус смартфона с узнаваемыми деталями: зона Dynamic Island и нижняя полоска Home. В «островке» расположена кнопка «ТЕМА», поэтому смена оформления находится на видном месте.

Цвет помогает читать карту блоков: опыт подсвечен розовым, учёба - синим, навыки - оранжевым, SaaS - зелёным. Даты двух мест работы дополнительно различаются песочным и виноградным тонами, а сами карточки слегка тонированы фоном.

На узких экранах media-запросы меняют сетку навыков, укорачивают длинные подписи и перестраивают шапку. Так выдерживается логика mobile-first.

3 ПРАКТИЧЕСКАЯ РЕАЛИЗАЦИЯ И РАЗВЁРТЫВАНИЕ

3.1 Структура проекта и основные компоненты

Собранный результат - набор статических файлов:

- `index.html` содержит разметку, стили и клиентские сценарии;
- `scr/photo/me.jpg` хранит портрет;
- каталог `scr/fav/` включает фавиконки и `web-manifest`;
- `robots.txt`, `sitemap.xml`, `.htaccess`, `404.html` обслуживают публикацию.

Соединение представления и логики в одном `index.html` ускоряет перенос: достаточно скопировать каталог в публичную область хостинга. Для учебного объёма это приемлемо; в промышленной практике стили и скрипты чаще выносят и собирают отдельно.

Среди заметных решений:

1. палитры через CSS-переменные и атрибут `data-theme`;
2. запоминание темы в `localStorage` ключом `kyr2-theme`;
3. подсчёт стажа из `data-start` формата `ММ.YYYY` с русскими формами слов;
4. поочерёдная подсветка навыков, которая замирает при наведении;
5. SEO-описания и JSON-LD сущностей `Person` и `WebPage`.

3.2 Взаимодействие с операционной средой хостинга

Ресурс доступен по адресу `http://trifonix.beget.tech/`. Площадка Beget даёт окружение Linux и Apache. Через `.htaccess` заданы:

- стартовый документ `index.html`;
- базовые security-заголовки;
- сжатие текстовых ответов;
- сроки кэша для картинок и HTML.

С позиции ОС это настройка уровня пользователя виртуального хоста: ядро системы не администрируется, но правила раздачи файлов контролируются. Подобная модель обычна для shared- и облачных площадок.

Файл `robots.txt` сообщает роботам границы обхода, а `sitemap.xml` указывает канонический адрес главной. Вместе с мета-описаниями это укрепляет техническую готовность страницы к индексации.

3.3 Клиентское хранение настроек и вычисляемые данные

Смена темы не ходит на сервер. При открытии страницы сценарий читает запись из `localStorage` и применяет палитру до показа основного интерфейса, чтобы уменьшить вспышку «чужого» цвета. Нажатие «ТЕМА» обновляет и экран, и сохранённое значение.

Срок работы считается в браузере по системной дате устройства. Для верхней карточки взята точка старта 04.2026, для нижней - 03.2025. Подпись выводится под «н.в.» двумя строками: годы и месяцы. Так демонстрируется работа со временем без СУБД: исходные даты лежат в разметке, а результат появляется в момент открытия.

Методически это подчёркивает простую мысль: не каждое вычисляемое поле обязано жить в таблице. Атрибуты HTML и клиентский расчёт уместны, пока нет нужды синхронно менять сведения для множества посетителей из одного центра.

3.4 Тестирование и результаты

Контроль включал:

- просмотр на широком и узком экране;
- проверку смены темы и её сохранения после обновления;
- сверку подсчёта стажа;
- работоспособность контактов и кнопки темы;
- наличие иконок вкладки и открытие боевого URL.

В итоге получена рабочая визитка с актуальными сведениями об авторе. Она фиксирует практику вёрстки, клиентских сценариев и выкладки и может служить и портфолио-элементом, и заготовкой для следующих персональных страниц.

ЗАКЛЮЧЕНИЕ

В курсовой работе рассмотрена разработка сайта-визитки фуллстек-разработчика в контексте современных операционных систем и подходов к хранению данных. Основные результаты состоят в следующем.

1. Показано, что сайт-визитка является эффективным средством профессиональной самопрезентации и может служить демонстрацией компетенций веб-разработчика.
2. Установлено, что даже статический веб-ресурс опирается на клиент-серверную модель и сервисы операционной системы хостинга: файловую систему, веб-сервер, сетевой стек и политики доступа.
3. Проанализированы варианты хранения данных. Для контента визитки достаточно файлового размещения; для пользовательских настроек интерфейса целесообразно клиентское хранилище `localStorage`; серверная СУБД требуется при появлении многопользовательской динамики и централизованного управления контентом.
4. Спроектирован и реализован одностраничный адаптивный сайт с блоками опыта, образования, навыков и проекта, а также с переключением темы и автоматическим расчётом стажа.
5. Выполнено развёртывание на хостинге Beget по адресу `http://trifonix.beget.tech/`, подготовлены файлы SEO и конфигурации веб-сервера.

Практическая значимость подтверждается наличием опубликованного результата. Дальнейшее развитие может включать серверный API, базу данных проектов, многоязычность и сбор обезличенной статистики посещений.

Список использованных источников

1. Никсон Р. Создаём динамические веб-сайты с помощью PHP, MySQL, JavaScript, CSS и HTML5. - СПб.: Питер, 2021. - 816 с.
2. Таненбаум Э., Бос Х. Современные операционные системы. - 4-е изд. - СПб.: Питер, 2020. - 1120 с.
3. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы. - СПб.: Питер, 2020. - 1008 с.
4. Дейт К. Дж. Введение в системы баз данных. - М.: Вильямс, 2019. - 1328 с.
5. Документация MDN Web Docs. Window.localStorage [Электронный ресурс]. - Режим доступа: <https://developer.mozilla.org/ru/docs/Web/API/Window/localStorage> (дата обращения: 10.09.2026).
6. Документация MDN Web Docs. HTML: HyperText Markup Language [Электронный ресурс]. - Режим доступа: <https://developer.mozilla.org/ru/docs/Web/HTML> (дата обращения: 10.09.2026).
7. Спецификация Schema.org. Person [Электронный ресурс]. - Режим доступа: <https://schema.org/Person> (дата обращения: 10.09.2026).
8. Хостинг Beget. Документация и справка [Электронный ресурс]. - Режим доступа: <https://beget.com/ru/kb> (дата обращения: 10.09.2026).
9. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приёмы объектно-ориентированного проектирования. Паттерны проектирования. - СПб.: Питер, 2020. - 366 с.
10. Фримен Э., Робсон Э. Изучаем HTML, CSS и JavaScript. - СПб.: Питер, 2021. - 720 с.